

System Architecture Design

How the platform is structured, how its parts interact, and why it is built the way it is

Item	Detail
Document	System architecture design (companion to the Backend Architecture Review)
Version	1.0
Date	10 October 2026
Scope	backend service, its data stores, integrations, deployment, and developer conventions
Basis	backend.zip: source, migrations, schema dump db.sql, design notes in docs/ and migrations/
Classification	Internal

Contents

- 1 Purpose and scope
- 2 Design goals and principles
- 3 System context
- 4 Architecture at a glance
- 5 Backend component design
- 6 Module map
- 7 Domain design
- 8 Key flows
- 9 Cross-cutting design
- 10 Integration design
- 11 Background processing and real time
- 12 Data design
- 13 Deployment architecture
- 14 Configuration design
- 15 Design decisions
- 16 Known design gaps
- 17 Glossary

1 Purpose and scope

This document describes the architecture of the Rippotai ERP backend as it exists today. It explains the components, how they are organised, how data moves through them, and the reasoning behind the main choices. It is written so that a new engineer can understand the system in a day and an architect can judge it against future requirements.

The platform runs the operating workflow of an architecture practice: from lead capture and project briefs through scope, bills of quantities, vendor selection, procurement, site operations, document control and handover, with a public client portal for approvals. It is delivered as one backend service with a staff web application and a client-facing portal as its consumers.

This document covers the backend and its immediate surroundings. The React frontend is referenced only where it shapes backend design. Security weaknesses are noted briefly in Section 16 and analysed in full in the separate Backend Architecture Review.

2 Design goals and principles

The design is guided by a small set of goals that can be read from the code and the design notes.

Goal	How the design serves it
Model the whole project lifecycle in one place	A single backend with one data store, so a project's brief, scope, BOQ, procurement and site records share identifiers and history.
Keep governance auditable	A gate engine decides when a project may advance. Transitions and overrides are recorded in history tables, with transactions and row locks.
Stay simple to deploy and reason about	A modular monolith deployed as one container, with Redis and MySQL as the only mandatory stateful dependencies.
Let staff work with live data quickly	Cached portfolio aggregates, real-time notifications, and search over the main entities.
Share limited views with outsiders safely	Opaque, expiring client links rather than client accounts.
Preserve commercial history	Versioned BOQs, quotations and estimates, and snapshot-based vendor comparison sheets.

Several principles recur across modules:

- **Uniform module shape.** Every feature module has the same layout: a module file, controllers, services, Sequelize models and DTOs. A developer who knows one module can navigate the others.
- **Validation at the edge.** Request bodies are validated against DTO classes before they reach business code. Unknown properties are dropped.
- **Business rules in services, not controllers.** Controllers translate HTTP into service calls. Services own transactions and invariants.
- **Derived data is never authoritative.** Dashboards, completion flags and search indices are caches. Decisions are made from primary records.
- **Honest degradation.** Where Redis is unavailable, the cache layer falls back to the database within a strict time budget.

3 System context

The backend sits between its staff and client users and a set of supporting and external systems. Figure 3.1 shows the context. Table 3.1 lists the interfaces.

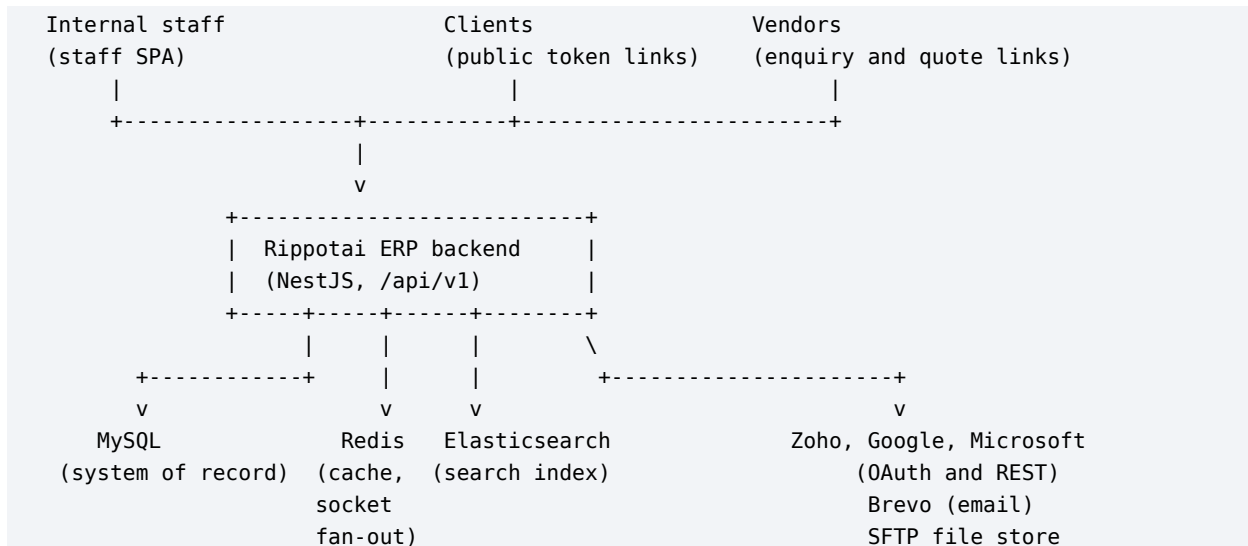


Figure 3.1. System context (text rendering).

Counterpart	Interface	Purpose
Staff web application	HTTPS REST with Bearer JWT; Socket.IO /notifications	All internal operations and live notifications
Client portal	HTTPS under /public/client/:token	Viewing and approving BOQs and quotations, accepting handover
MySQL	Sequelize over mysql2	Authoritative data for every module
Redis	ioredis; Socket.IO Redis adapter	Shared cache, socket fan-out, gate coordination
Elasticsearch	Official client	Global and entity search
Zoho	OAuth 2 and REST	CRM contacts and deals, project tasks, file storage, calendar, chat
Google, Microsoft	OAuth 2 and REST	Calendar, tasks, OneDrive
Brevo	SDK	Transactional email
SFTP file store or local directory	ssh2-sftp-client or Express static	Document and image storage

4 Architecture at a glance

The backend is a modular monolith. One Node.js process hosts about 37 feature modules that share a database and a common infrastructure layer. The choice favours a single deployable unit, transactional consistency across the project lifecycle, and simple operations for a small engineering team. The cost is that scaling is coarse, and a few responsibilities that would normally be asynchronous run inside the request process today.

Figure 4.1 shows the logical layers inside the service.

Transport	Express HTTP	Socket.IO /notifications	
Pipeline	ValidationPipe -> JwtAuthGuard -> PermissionsGuard -> controller -> interceptors (secret stripping)		
Feature modules (x37)	projects gates boqs material-procurement site-operations documents vendors search ... each: controller -> service -> Sequelize model		
Shared services	common/ (guards, decorators, Redis, mail, automation bus, enums, types, DTO validators)		
Infrastructure	MySQL (Sequelize) Redis Elasticsearch SFTP Puppeteer (PDF) external OAuth and REST APIs		

Figure 4.1. Logical layers of the backend (text rendering).

Three cross-cutting mechanisms connect the layers: a shared automation event bus for reacting to domain events, a Redis-backed Socket.IO adapter for fan-out, and a Redis-backed cache for read-heavy aggregates. Each is covered in later sections.

5 Backend component design

5.1 Module template

Each feature module is a folder under `src/modules` that exposes a single `*.module.ts`. The module registers its controllers, services and models with the framework and imports any shared modules it needs. Controllers are thin. Services contain the logic. Models describe persistence. DTOs describe request bodies and are validated on entry.

Part	Responsibility	Example
*.module.ts	Wiring and dependency declaration	<code>gates.module.ts</code>
*.controller.ts	Routes, guards, parameter extraction, delegation	<code>gates.controller.ts</code>
*.service.ts	Business rules, transactions, orchestration	<code>gate-engine.service.ts</code>
models/	Sequelize entity definitions and associations	<code>project_gates</code> , <code>gate_logs</code>
dto/	Request validation with class-validator	<code>create-project-brief.dto.ts</code>
Specs (*.spec.ts)	Unit tests colocated with the code	<code>gate-engine.spec.ts</code>

5.2 Request pipeline

Every request passes through the same sequence. This is the main consistency mechanism of the platform, and the place where the security boundary is meant to live.

1. Express parses the body with a 3 MB JSON limit, sized for saved comparison snapshots.
2. The global `ValidationPipe` whitelists and transforms the body against the DTO.
3. Guards run. `JwtAuthGuard` verifies the token and confirms the session is live. `PermissionsGuard` checks a named permission where it is applied. `CdnGuard` protects the internal upload endpoint.
4. The controller calls a service. Services use Sequelize transactions for multi-step writes.
5. The response passes through `StripSecretsInterceptor`, which removes password hashes, token hashes and OAuth tokens at any depth.
6. Errors are normalised by a shared HTTP exception filter.

The pipeline is applied per controller rather than globally. This is a design choice with a consequence: a controller that omits its guard is unprotected. The Review documents where this has happened.

5.3 Shared infrastructure in common/

Component	Role
Guards	<code>JwtAuthGuard</code> , <code>PermissionsGuard</code> , <code>PortalAccessGuard</code> , <code>CdnGuard</code>
Decorators	<code>CurrentUser</code> , <code>RequirePermission</code> , <code>OwnerKey</code> , http exception filter
Interceptors	<code>StripSecretsInterceptor</code>

Component	Role
Redis module	Shared ioredis client
Mail service	Brevo client wrapper
Automation bus	In-process EventEmitter carrying five domain event types
Enums and types	Shared status vocabularies (for example gate, approval, purchase order, quotation, site stage)
Validators	Custom class-validator rules, such as valid document section keys
Gateway	Notification Socket.IO gateway with token-authenticated rooms per user

6 Module map

The 37 modules group naturally into business domains. Table 6.1 lists each module with its approximate size and responsibility. Sizes are lines of TypeScript including specs and are indicative only.

Module	Lines	Controllers	Responsibility
material-procurement	12,764	12	Requirements, rate sheets, quotations, POs, work orders, challans, inventory
projects	10,048	7	Project directory, phases, planner, command center, dashboards
site-operations	8,867	13	Visits, daily site reports, QC checklists, snags, RFIs, mock-ups, DPR
boqs	5,497	4	BOQs, versions, approvals, templates, library, export
zoho	5,290	5	Zoho Bigin CRM, Projects, Cliq, WorkDrive, Calendar integrations
documents	4,962	6	Document catalogue, versions, drawings and revisions, requirements
vendors	4,213	7	Vendor directory, shortlists, shortlist packages, rate comparisons
engagement	4,181	2	Activity logs and notifications, Socket.IO fan-out
search	3,938	1	Elasticsearch global, entity and autocomplete search
users	3,910	3	Users, teams, signatures, avatars
quotations	2,835	3	Quotations, items, versions, comparison
brief	2,717	1	Project briefs and their child tables
gates	2,676	2	Gate engine: conditions, evidence, transitions, overrides
budget-estimate	2,617	1	Budget estimates and versions
leads	2,470	1	Leads, notes, tasks, activity (Zoho-linked)
auth	2,305	6	Login, signup, sessions, OAuth (Google, Microsoft, Zoho), verification tokens
reki	2,197	1	Site reconnaissance (site recces), rooms and photos
scope-of-work	1,957	1	Scope of work, spaces and items
process-workflow	1,764	5	Process library, progress, timeline, continuity, gates controller
sync	1,735	1	Zoho reminder sync and sync logs
payments	1,284	1	Payment schedules and milestones
automation	1,223	1	Automation rules, engine, triggers, audit
rbac	1,219	5	Roles, permissions, role-app and role-permission mappings
plan-of-action	1,213	1	Plans of action and phases
dashboard	1,145	1	Dashboard widgets and layouts

Module	Lines	Controllers	Responsibility
google	1,032	2	Google Calendar and Tasks
microsoft	1,024	1	Microsoft OneDrive and calendar
tasks	939	1	Tasks and task definitions
metas	789	2	Units and terms-and-conditions templates
client-portal	772	1	Client links (public token), BOQ approval by client, handover
calendar	563	1	Calendar events
workspace	542	2	Workspace notes and tasks
reports	313	1	Read-only aggregate reports
clients	312	1	Client directory
settings	216	1	Application and company settings
cdn	204	1	File upload to SFTP CDN or local static directory
business-proposals	192	1	Business proposals

Table 6.1. Module inventory.

6.1 Domain grouping

Domain	Modules	What the domain owns
Identity and access	auth, users, rbac, settings	Sign-in, sessions, users and teams, roles, permissions and app access
Clients and CRM	clients, leads, reki, brief, sync	Client records, lead pipeline, site reconnaissance, project briefs, CRM synchronisation
Project delivery	projects, gates, process-workflow, plan-of-action, tasks, calendar, scope-of-work, workspace	Project records and phases, the gate engine, delivery workflows, planning and tasks
Commercial	boqs, quotations, budget-estimate, payments, business-proposals, metas	Bills of quantities, quotations, budgets, payment schedules, proposals, templates
Procurement and vendors	material-procurement, vendors	Materials, requirements, rate sheets, purchase and work orders, challans, inventory, shortlists
Documents and drawings	documents (includes drawings), cdn	Document catalogue, versions, drawing register, file storage
Site operations	site-operations	Visits, daily reports, QC checklists, snags, RFIs, mock-ups, DPR documents
Control and insight	automation, engagement, dashboard, reports, search, sync	Rules, notifications, activity logs, dashboards, reporting, search

Domain	Modules	What the domain owns
External integrations	zoho, google, microsoft, auth/oauth	Connections to third-party systems and their token lifecycles
Public access	client-portal	Tokenised client views and actions

7 Domain design

The data model has 164 tables. Most business tables use UUID keys and timestamps. Soft deletion is used on 41 models. The model is organised around a few aggregates, each with a clear root and children.

Aggregate	Root	Design intent
Project	projects	Anchor for everything delivered for a client. Holds phases, locations, scope categories and planner links. Its progress fields are display caches.
Gate	project_gates, gate_definitions	Governance checkpoints. Twelve gates are defined. Each has conditions (documents, drawings, QC, tasks, payments) and a history of transitions.
Client brief	project_briefs	Structured requirements from the client, with eleven child tables for phases, spaces, services, references and restrictions.
BOQ	boqs	Itemised cost and scope, versioned, with categories, items, miscellaneous lines and an approval workflow.
Quotation	quotations	Vendor or material quotes with items and versions, and comparison results.
Vendor rate comparison	vendor_rate_comparisons	A saved snapshot of BOQ lines and vendor rates, with a revision counter so that edits do not overwrite each other.
Procurement	material_requirements	Flow from requirement to quotation, purchase order, work order, delivery challan and inventory movement.
Document	documents, drawings	Catalogue of 83 document types, versions, attachments, and drawing revisions. Gate evidence is derived from the latest submission.
Site operations	architect_site_visits, daily_site_reports, quality_checklists, snag_lists, rfis	Field records linked to the project by UUID.
Payment	payment_schedules	Milestones that feed the gate engine and overdue alerts.
Client link	client_links	Hashed lookup token, encrypted copy for redisplay, purpose, target and expiry.
Identity	users, roles, auth_tokens	Accounts, role assignments and live session records.

7.1 Governance: the gate engine

The gate engine is the design's central control. A project moves through gated phases, and clearing a gate requires that all of its conditions are satisfied from primary records. The engine locks the project row during a clear or reopen, evaluates conditions, writes the transition in the same transaction, and relocks

downstream gates on reopen. Overrides are allowed only where a gate permits them, and require a written reason. Completion flags on the project are explicitly not evidence.

7.2 Commercial history

Commercial documents are never edited in place once approved. BOQs, quotations and estimates keep version rows, so the history of a price or scope change is preserved. Vendor rate comparison sheets store a complete snapshot, which means later BOQ changes do not silently alter a comparison that was used for a decision.

7.3 Phase vocabularies

Three phase vocabularies coexist: DOCUMENTS phases for the gate engine, consultancy and PMC phases for the planner, and process-workflow phases for site operations. Each feature uses one vocabulary. Developers should identify which one applies before reading a phase identifier.

8 Key flows

8.1 Authentication and sessions

7. Login verifies the email and bcrypt password hash. The service creates a random session identifier and stores only its SHA-256 hash.
8. A JWT carrying the identifier is signed with the server secret and returned. It expires in seven days.
9. On each guarded request, the token signature and expiry are checked, and the hash of its identifier must match a live session row. Revoking the row ends the session immediately.

8.2 Gate clearance

10. A clear request is authorised against gates:clear.
11. Inside a transaction the project row is locked and the gate's conditions are evaluated against documents, drawings, QC, tasks and payments.
12. On success the gate is cleared, the next gate advances, and a transition record is written. The Command Center cache is invalidated after commit.

8.3 Commercial approval

A BOQ is submitted for approval, approved, and may be duplicated into a new version when scope changes. The approver is recorded in the approval history. A client can approve the BOQ through a client link, which records the action against the link and the project.

8.4 Client portal

A staff user creates a client link with a purpose and an expiry. The client opens the link, which is hashed and matched, then sees only the view or action that the link allows. Actions are recorded. The client never receives a staff session.

8.5 Search

Global search builds an Elasticsearch query that applies a project filter for non-administrators and excludes deleted records unless explicitly requested by an administrator. Entity search and autocomplete share the same permission model. The index is populated by a bulk reindex operation rather than by live updates.

8.6 Notifications

Domain changes that matter to a user produce notification records. Those records are pushed over the notification socket to the user's private room. Clients connect with their token, which is verified on connection, and the Redis adapter carries messages between replicas outside development.

8.7 Automation

Five domain events (QC failure, overdue payment milestone, overdue task, stalled phase, overdue RFI) feed an in-process bus. A rule engine evaluates active rules on a fixed interval and on events. Matching rules create notifications or escalations and write an audit trail.

8.8 Portfolio aggregation

The Command Center builds one portfolio aggregate and serves its panels from it. Results are fresh for 15 seconds, a stale snapshot is served while a single rebuild runs, and Redis holds the result for 120 seconds. Writes to the underlying tables rotate a namespace version so that stale aggregates are not reused. The cache is never consulted for authorisation or gate decisions.

9 Cross-cutting design

Concern	Design
Authentication	Session JWT with server-side revocation. Passwords hashed with bcrypt at cost 10. Password reset tokens are random, hashed, single-use and expire after one hour.
Authorisation	Role-based permissions in resource:action form, grouped into app access per role. Enforced by PermissionsGuard on the controllers where it is applied. Object-level checks exist in search and the gate engine.
Validation	DTOs with class-validator on every request body. Whitelisting strips unknown fields.
Persistence and transactions	Sequelize with explicit transactions for multi-table writes. Row locks for state machines. Revision counters for shared sheets.
Secrets	Response serialisation removes secret columns. Third-party tokens and session hashes never leave the server.
Errors	A shared exception filter converts exceptions into consistent HTTP responses.
Logging	The Nest Logger is used across modules. Structured logging is not yet in place.
Caching	Redis for the Command Center aggregate with generation-based invalidation and strict time budgets. Database fallback on failure.
Configuration	Environment variables read from process.env and ConfigService. Values are described in the Review's appendix.
Time and dates	Stored as database timestamps and returned as ISO 8601 strings.

10 Integration design

External systems are reached through module-local services. Each integration owns its tokens and its client. The pattern is pragmatic and keeps each provider isolated, at the cost of some repeated code for token refresh and error handling.

Integration	Pattern	Role in the design
Zoho Bigin (CRM)	OAuth 2 authorization code; REST	Source of leads and deals. Synchronised into the lead pipeline.
Zoho Projects	REST	Mirror of project tasks and reminders.
Zoho Cliq, WorkDrive, Calendar	REST	Team messaging, file storage and calendar links.
Google Calendar and Tasks	OAuth 2	Calendar events and task lists for staff.
Microsoft OneDrive and calendar	OAuth 2	File links and calendar.
Brevo	SDK	Transactional email for verification, reset and notifications.
SFTP file store	ssh2-sftp-client	Durable file storage. Falls back to a local directory served at /cdn when no host is configured.
Puppeteer	In-process headless Chromium	Renders BOQ and document PDFs from HTML templates.

OAuth flows use a state parameter signed with a separate secret and expiring after ten minutes. Provider tokens are stored with the user they belong to.

11 Background processing and real time

Three mechanisms run outside the request cycle. They are simple by design and currently run inside the API process.

Mechanism	Design	Note
Automation engine	Evaluates active rules on a timer (default every 30 minutes) and on domain events	Timer and bus live in the process. A durable queue is the intended evolution.
Reminder sync	Polls Zoho reminders every minute	Runs in each process that starts the module.
Notification socket	Per-user rooms over Socket.IO, token-authenticated	The Redis adapter provides fan-out between replicas outside development.

The design intent is that event handling becomes durable: domain changes write an outbox row in the same transaction, and a queue delivers it to the automation engine, notifications and search. That intent is not implemented yet.

12 Data design

12.1 Storage

- **MySQL** is the system of record, accessed through Sequelize with schema synchronisation disabled. Schema changes are made by hand-written SQL files in migrations/, each with a design note covering verification and rollback.
- **Redis** holds shared cache entries and socket fan-out state. It is not a system of record.
- **Elasticsearch** holds a denormalised search copy of selected entities across 16 indices. It is rebuilt rather than kept live.

12.2 Identifiers

New tables use UUIDs stored as CHAR(36). Some older site operations and process-workflow tables still use integers and are mapped where they meet UUID-keyed data. The design intent is to standardise on UUIDs.

12.3 Concurrency

Pattern	Where
Pessimistic row lock in a transaction	Gate clear and reopen
Revision counter	Vendor rate comparison sheets
New version row	BOQs, quotations, budget estimates
Last write wins	Most other updates

12.4 Soft deletion and history

Business records are soft-deleted through a paranoid model. Transient tokens are hard-deleted. History is kept in dedicated tables, such as gate transitions and BOQ activities, and in a shared activity log.

13 Deployment architecture

The service is packaged as a single container image built from the backend folder. The image runs the compiled entry point on port 5000. A Compose file runs the API alongside Redis for local and single-host deployment. Production runs the same image on a virtual private server, updated by a continuous delivery workflow that publishes to a container registry and restarts the service over SSH.

Element	Design
Image	Node 20 on Alpine, with Chromium installed for PDF rendering. Built with npm ci and nest build.
Local and single-host	docker-compose with the API and Redis. Redis data persists in a named volume.
Production	Container on a VPS, behind a reverse proxy that terminates TLS (proxy configuration is outside the repository).
Delivery	Build and push on changes under backend/, then pull and recreate the service over SSH.
Environments	Behaviour switches on NODE_ENV. Development disables the Redis socket adapter and enables verbose SQL logging.
Data stores	MySQL, Redis and Elasticsearch are provided as external services and configured by environment variables.

Design note. Because behaviour depends on NODE_ENV, the environment name is a functional switch, not just a label. Any new environment must set it deliberately.

14 Configuration design

Configuration is supplied through environment variables, loaded from a .env file in development and from the container environment in production. About 25 variables are read across the code, covering database access, session and OAuth secrets, Redis, Elasticsearch, the file store, Zoho, public URLs and automation timing. Appendix B of the Review lists them. The design intent is one typed schema validated at startup; today values are read in several places without a single validation step.

15 Design decisions

The decisions below are inferred from the code and design notes, since the repository does not hold formal decision records. Each should be confirmed by its owner.

ID	Decision	Rationale	Trade-off
AD-01	Modular monolith	One team, a tightly coupled domain, simple operations	Coarse scaling; in-process coupling
AD-02	MySQL with manual SQL migrations	Existing data; production safety	Migrations are not tracked by the application
AD-03	Session JWT with database revocation	Immediate logout and revoke-all	A database lookup per request
AD-04	Gate engine as the authority for progression	Auditable governance	All progression must go through the engine
AD-05	Pessimistic row locks for gates	No racing clears or reopens	Serialises work per project
AD-06	Multi-index Elasticsearch	Per-entity mapping and ranking	Needs a sync pipeline that is not yet built
AD-07	Redis for cache and socket fan-out	Shared state between replicas	Adapter depends on NODE_ENV
AD-08	Opaque tokens for client access	No client accounts to manage	Key handling must be strong
AD-09	In-process automation bus and timers	Simplicity	Events are not durable; timers repeat per replica
AD-10	Versioned commercial documents	Traceable pricing and scope history	Storage growth; version selection care
AD-11	Headless Chromium for PDFs	Exact layouts from HTML templates	Heavy resource use in the API process

16 Known design gaps

The design is coherent, but several intended properties are not yet met. The most important are listed here. Each is traced to evidence and remediation in the Backend Architecture Review, findings register.

Gap	Design consequence	Review reference
Authentication is not applied to most routes	The intended boundary between anonymous and authenticated access is not enforced by the framework	SEC-01
Permissions are enforced in one module	Role design exists but does not govern most actions	SEC-02
Search index is not kept current	Search reflects the last rebuild, not the live data	ARCH-03
Scheduled work and events are not durable or replica-safe	Duplicate or lost automation under scale or restart	ARCH-01, ARCH-02
Readiness does not test dependencies	Load balancers cannot detect a failed database	ARCH-08
Schema changes are not tracked by the application	Production state is not self-describing	ARCH-07
Configuration is not validated at startup	Misconfiguration surfaces at runtime	ARCH-09

The remediation sequence in the Review's roadmap is designed to close these gaps in order of risk: containment and authorisation first, then durable background processing and observability.

17 Glossary

Term	Meaning
Aggregate	A cluster of related records with one root that controls changes to them.
BOQ	Bill of quantities: itemised scope and cost for a project.
Client link	An opaque, expiring token that gives a client a specific view or action.
Command Center	Portfolio dashboard built from an aggregate and cached.
DPR	Daily progress report from site.
Gate	A governed checkpoint in a project's progression.
Outbox	A table of domain events written in the same transaction as the change, then delivered to consumers.
Paranoid model	A model that soft-deletes rows instead of removing them.
Shortlist package	A saved vendor and material selection that can be applied to another project.
Socket room	A named group of connections that receives messages for one user.